

DATA STORAGE & MANAGEMENT

He thong Dang ky Hoc phan & Thu Hoc phi

Ly thuyet nen tang | Thuc hanh tren MySQL | So sanh MongoDB

Data Models	Bang quan he, Collection, Schema
Storage Engine	InnoDB vs WiredTiger
CRUD & SQL	8 loai query theo spec
Schema Design	FK, ENUM, Constraint, Index

Muc luc

Chuong	Noi dung	Trang
1	Khai niem nen tang — Data Storage & Management la gi?	2
2	Mo hinh du lieu — Relational vs Document	3
3	Storage Engine — InnoDB va WiredTiger	4
4	Schema thuc te — 5 bang MySQL va 5 collections MongoDB	5
5	8 loai truy van — ap dung vao he thong	6

6	Chung minh MySQL phu hop hon — 7 tieu chi	8
7	Ket luan	9

CHUONG 1

Khai niem Nen tang

Data Storage & Management la gi? Tai sao quan trong?

1.1 Dinh nghia

Data Storage & Management (DSM) la tap hop cac ky thuat, cong nghe va quy tac de luu tru, to chuc, truy cap va duy tri du lieu mot cach hieu qua, nhat quan va an toan. DSM khong chi la viec "cho du lieu vao cho nao" ma con bao gom ca cach structurate du lieu, cach dam bao tinh toan ven (integrity), cach toi uu truy xuat va cach bao ve du lieu khi co su co.

Trong he thong dang ky hoc phan nay: DSM quyet dinh cach luu Users, Classes, Enrollments va SystemState; cach dam bao sinh vien khong the dang ky trung lop; cach admin reset du lieu cuoi hoc ky ma khong mat thong tin tai khoan; va cachphan quyen theo 4 giai doan he thong.

1.2 4 nhien vu cot loi cua DSM

Luu tru (Storage):

Chon he thong luu tru phu hop: RDBMS, NoSQL, file system, in-memory. Quyet dinh den hieu nang, kha nang mo rong va chi phi.

To chuc (Organization):

Dinh nghia schema, cau truc bang/collection, kieu du lieu, rang buoc (constraints). Quyet dinh den do nhat quan va de mo rong cua he thong.

Truy cap (Access):

Cung cap ngon ngu truy van (SQL/MQL), API va index de tim kiem du lieu nhanh. Quyet dinh den toc do phan hoi cua ung dung.

Bao tri (Maintenance):

Dam bao du lieu luon chinh xac, backup dinh ky, phuc hoi khi co su co va dieu chinh hieu suat theo thoi gian.

1.3 Phan biet Data Storage va Database Management

Data Storage = phan cung/phan mem luu tru vat ly (SSD, HDD, RAM, cloud storage).

Database Management System (DBMS) = phan mem quan ly viec doc/ghi/truy van du lieu mot cach co to chuc. MySQL va MongoDB deu la DBMS. He thong nay dung MySQL lam DBMS chay tren phan cung cua server (hoac container Docker).

1.4 Vai tro cua DSM trong he thong dang ky hoc phan

He thong dang ky hoc phan co 4 giai doan trang thai (Stage), 3 role nguoi dung (Admin/Teacher/Student) va 8 loai truy van bat buoc. Moi quyet dinh DSM deu co anh huong truc tiep den tinh nhut quan va tinh dung dan cua nghiep vu:

Quyết định DSM	Ảnh hưởng nghiệp vụ	Thực hiện
Chọn MySQL làm RDBMS	Đảm bảo ràng buộc FK, ENUM, CASCADE	Schema 5 bảng
ENUM current_stage cho	Hệ thống không thể có stage sai	SystemState.current_stage
UNIQUE enrollment KEY	Sinh viên không thể đăng ký trùng lớp	UNIQUE(student_id,class_id)
FK + CASCADE	Reset học kỳ sạch hoàn toàn trong 1 lệnh	ON DELETE CASCADE
FK teacher_id -> Users	Tên GV luôn mới nhất, không bị lỗi	JOIN qua FK

Mo hình Du lieu

Relational Model vs Document Model — Chon cai nao?

2.1 Mo hình Quan he (Relational Model) — MySQL

Mo hình quan he to chuc du lieu thanh cac **bang (table)** voi hang (row) va cot (column). Moi bang co mot schema co dinh. Cac bang lien ket qua **Foreign Key (FK)**. Ngon ngu truy van la SQL (Structured Query Language).

Schema co dinh (Fixed Schema):

Moi cot co kieu du lieu xac dinh truoc. MySQL tu choi du lieu sai kieu. Vi du: credits INT — khong the luu "ba" vao day.

Rang buoc toan ven (Constraints):

PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, ENUM dam bao du lieu hop le tai tang DB — khong can code xu ly.

Tinh nhât quan (ACID):

Atomicity, Consistency, Isolation, Durability — moi transaction hoac thanh cong hoan toan, hoac rollback hoan toan.

Chuan hoa (Normalization):

Tach du lieu de tranh trung lap. Ten mon hoc chi luu 1 lan trong Courses, cac bang khac tham chieu qua course_code.

2.2 Mo hình Document (Document Model) — MongoDB

Mo hình document luu du lieu thanh cac **document (JSON/BSON)** ben trong collection. Moi document co the co cau truc khac nhau (schema-less hoac schema-flexible). Ngon ngu truy van la MQL (MongoDB Query Language).

Uu diem cua Document Model: Linh hoạt trong cau truc du lieu, phu hop cho du lieu thay doi thuong xuyen. Scale ngang (horizontal sharding) de dang hon RDBMS. Anh xa tu nhien voi cau truc object trong code (ORM khong can thiet). **Nhuoc diem:** Khong co native FK, ENUM, CASCADE. Moi rang buoc phai xu ly o application layer. JOIN (goi la \$lookup) cham hon va verbose hon SQL JOIN.

2.3 So sanh truc tiep — Ung dung vao he thong

Dac diem	MySQL (Relational)	MongoDB (Document)
Kieu du lieu	INT, VARCHAR, TIME, ENUM — strict	BSON: String, Number, Date — flexible
Rang buoc	FK, UNIQUE, NOT NULL tai DB layer	Chi co unique index,phan con lai o app

Quan he bang	JOIN native, 1 cau SQL, co index	\$lookup pipeline, verbose, cham hon
Schema stage	ENUM("create_class",...) — DB validate	String tự do — typo không bị bắt
Reset hoc ky	1 lenh TRUNCATE + CASCADE tự động	2 lenh riêng, có thể crash giữa chúng
Scale ngang	Kho hon (sharding phức tạp)	De han (built-in sharding)
Phu hop he thong nay?	Rat phu hop	Được nhưng cần nhiều workaround

Storage Engine

InnoDB (MySQL) và WiredTiger (MongoDB) — Cơ chế lưu trữ vật lý

3.1 InnoDB — Storage Engine của MySQL

InnoDB là storage engine mặc định của MySQL từ version 5.5+. Đây là lý do MySQL được chọn cho hệ thống này: InnoDB hỗ trợ đầy đủ ACID transactions, row-level locking, foreign key constraints và crash recovery tự động.

B+ Tree Index:

Dữ liệu và index đều được lưu trên cấu trúc B+ Tree. Primary key = clustered index (dữ liệu nằm chính trong node lá của B+ tree). Secondary index lưu pointer đến PK.

Buffer Pool:

Cache trạng thái dữ liệu trong RAM (mặc định 128MB, nên tăng lên 70-80% RAM). Giảm I/O đĩa cứng. Với hệ thống học phân, buffer pool giúp query TKB nhanh sau lần đầu.

Redo Log / Undo Log:

Redo log đảm bảo dữ liệu không mất khi crash (Write-Ahead Logging). Undo log cho phép rollback transaction và MVCC (Multi-Version Concurrency Control).

Row-level Locking:

Chỉ khóa dòng đang đọc/ghi, không khóa cả bảng. Quan trọng khi nhiều sinh viên đăng ký cùng lúc trong Stage register_class.

Foreign Key Engine:

InnoDB là engine duy nhất của MySQL hỗ trợ FOREIGN KEY. Nếu dùng MyISAM, FK sẽ bị bỏ qua. Hệ thống này bắt buộc phải dùng InnoDB.

Kiểm tra InnoDB

```
-- Kiểm tra storage engine của các bảng
SELECT table_name, engine
FROM information_schema.tables
WHERE table_schema = "course_registration";

-- Kết quả mong đợi:
-- Courses | InnoDB
-- SystemState | InnoDB
-- Users | InnoDB
-- Classes | InnoDB
-- Enrollments | InnoDB
```

3.2 WiredTiger — Storage Engine của MongoDB

WiredTiger là storage engine mặc định của MongoDB từ version 3.2+. Sử dụng B+ Tree cho index và document storage. Hỗ trợ compression (snappy/zlib), document-level locking và MVCC từ version 4.0.

So sánh thực tế cho hệ thống này: InnoDB lưu Enrollments(student_id, class_id) với clustered index trên PK — lookup theo student_id + class_id rất nhanh. WiredTiger lưu document enrollments với ObjectId làm _id — cần compound index riêng cho { student_id, class_id } để đạt tốc độ tương đương. InnoDB có lợi hơn vì FK constraint được built-in, WiredTiger cần index thủ công.

3.3 Luồng đọc/ghi dữ liệu — Từ SQL đến đĩa

Khi sinh viên nhận "Đăng ký môn học", luồng xử lý trong MySQL/InnoDB như sau:

Bu oc	Tang xử lý	Mô tả
1	Application	Nhận HTTP POST /enroll {student_id, class_id}
2	SQL Parser	Parse câu INSERT INTO Enrollments VALUES (...)
3	Query Optimizer	Chọn execution plan tối ưu (index nào dùng)
4	InnoDB Buffer	Kiểm tra buffer pool — nếu có -> đọc từ RAM
5	Lock Manager	Đặt row-level lock trên dòng cần ghi
6	Redo Log	Ghi WAL (Write-Ahead Log) trước khi ghi data
7	Data Write	Ghi dữ liệu vào B+ Tree trên đĩa (hoặc buffer)
8	FK Check	Verify student_id và class_id tồn tại trong Users/Classes
9	Unique Check	Verify UNIQUE(student_id, class_id) chưa tồn tại
10	Commit	Tra kết quả thành công hoặc rollback nếu vi phạm

CHUONG 4

Schema Thuc te

5 bang MySQL & 5 collections MongoDB — Song song, giai thich tung truong

4.1 Bang Courses (Static Data)

MySQL	MongoDB
<pre>CREATE TABLE Courses (course_code VARCHAR(20) PRIMARY KEY, course_name VARCHAR(255) NOT NULL, credits INT NOT NULL, department VARCHAR(100) NOT NULL); -- Static seed, KHONG xoa khi reset hoc ky -- course_code la FK trong Classes</pre>	<pre>// Collection: courses { _id: "CS202", course_name: "Co so Du lieu", credits: 3, department: "Cong nghe thong tin" } // _id = course_code de lam ref</pre>

Ly do khong xoa Courses khi reset: Courses la du lieu tinh (static master data). Danh sach mon hoc khong thay doi theo tung hoc ky. Chi Classes va Enrollments moi duoc reset. Trong MySQL, viec giu Courses trong khi xoa Classes duoc thuc hien tu dong nho FK: Classes -> Courses (ON DELETE RESTRICT mac dinh cho Courses).

4.2 Bang SystemState (Singleton)

MySQL — ENUM dam bao stage hop le	MongoDB — String tu do, de bi typo
<pre>CREATE TABLE SystemState (id INT PRIMARY KEY DEFAULT 1, current_stage ENUM('create_class', 'register_class', 'lock_class', 'scheduled_class') NOT NULL DEFAULT "create_class", updated_at TIMESTAMP DEFAULT NOW() ON UPDATE CURRENT_TIMESTAMP);</pre>	<pre>// Collection: system_state (singleton) { _id: 1, current_stage: "create_class", // Chi la String – KHONG co validation! // Phai validate o application: // const VALID = ["create_class", // "register_class", "lock_class", // "scheduled_class"]; updated_at: ISODate("2024-09-01") }</pre>

4.3 Bang Users

MySQL	MongoDB
<pre>CREATE TABLE Users (person_id VARCHAR(50) PRIMARY KEY, email VARCHAR(255) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, full_name VARCHAR(255), role ENUM("admin","teacher","student") NOT NULL DEFAULT "student", created_at TIMESTAMP DEFAULT NOW()); -- Dang nhap bang email HOAC person_id -- UNIQUE tren email dam bao o DB level</pre>	<pre>// Collection: users { _id: "SV001", // person_id email: "sv001@student.edu.vn", password: "\$2b\$10\$...", full_name: "Nguyen Van A", role: "student", created_at: ISODate("2024-09-01") } // Index bat buoc: db.users.createIndex({email:1},{unique:true})</pre>

4.4 Bang Classes — Trung tam cua he thong

MySQL — 2 FK tu dong validate	MongoDB — Ref bang string, khong auto check
<pre>CREATE TABLE Classes (class_id INT AUTO_INCREMENT PRIMARY KEY, course_code VARCHAR(20) NOT NULL, teacher_id VARCHAR(50) NOT NULL, day_of_week VARCHAR(100) NOT NULL, start_time TIME NOT NULL, end_time TIME NOT NULL, created_at TIMESTAMP DEFAULT NOW(), FOREIGN KEY (course_code) REFERENCES Courses(course_code) ON DELETE CASCADE, FOREIGN KEY (teacher_id) REFERENCES Users(person_id) ON DELETE CASCADE);</pre>	<pre>// Collection: classes { _id: ObjectId("64f3a..."), course_code: "CS202", teacher_id: "T001", // KHONG luu teacher_name o day // Lay qua \$lookup de tranh lenh day_of_week: "Monday", start_time: "07:30", end_time: "09:30", created_at: ISODate("2024-09-01") } db.classes.createIndex({teacher_id:1}) db.classes.createIndex({course_code:1})</pre>

Quan trọng — Phan biet class_id va course_code: class_id la khoa chinh cua mot LOP HOC cu the (1 giang vien, 1 lich cu the). course_code la ma MON HOC (co the co nhieu lop cung 1 mon). Vi du: Mon CS202 co the co Class #1 (teacher T001, Thu 2) va Class #2 (teacher T002, Thu 5). Sinh vien dang ky class_id, khong phai course_code.

4.5 Bang Enrollments — Vong doi dang ky

MySQL — constraint tai DB	MongoDB — constraint qua index
---------------------------	--------------------------------

```

CREATE TABLE Enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(50) NOT NULL,
  class_id INT NOT NULL,
  status ENUM('enrolled','payed')
  NOT NULL DEFAULT 'enrolled',
  payment_requested BOOLEAN DEFAULT FALSE,
  enrolled_at TIMESTAMP DEFAULT NOW(),
  UNIQUE KEY unique_enroll
  (student_id, class_id),
  FOREIGN KEY (student_id)
  REFERENCES Users(person_id) ON DELETE
  CASCADE,
  FOREIGN KEY (class_id)
  REFERENCES Classes(class_id) ON DELETE
  CASCADE
);

```

```

// Collection: enrollments
{
  _id: ObjectId("64f4b..."),
  student_id: "SV001",
  class_id: ObjectId("64f3a..."),
  status: 'enrolled',
  payment_requested: false,
  enrolled_at: ISODate("2024-09-05")
}

// Index bat buoc (unique compound):
db.enrollments.createIndex(
  { student_id: 1, class_id: 1 },
  { unique: true }
)

db.enrollments.createIndex({class_id:1})
db.enrollments.createIndex({student_id:1,status:1})

```

Vong doi trang thai Enrollment: not-enrolled (khong co ban ghi) → **enrolled** (INSERT khi SV dang ky) → **payed** (UPDATE khi Admin xac nhan) → **bi xoa** (DELETE CASCADE khi reset hoc ky). MySQL ENUM dam bao chi 2 gia tri "enrolled" va "payed" ton tai — khong the INSERT gia tri nao khac vao cot status.

CHUONG 5

8 Loai Truy Van

INSERT / DELETE / UPDATE / SELECT — Ap dung vao he thong, so sanh MySQL va MongoDB

a. INSERT — Them du lieu moi

3 truong hop INSERT chinh trong he thong:

MySQL	MongoDB
<pre>-- 1. Sinh vien dang ky tai khoan INSERT INTO Users (person_id, email, password, full_name, role) VALUES (?, ?, ?, ?, 'student'); -- 2. Teacher tao lop moi (Stage 1) INSERT INTO Classes (course_code, teacher_id, day_of_week, start_time, end_time) VALUES (?, ?, ?, ?, ?); -- 3. Sinh vien dang ky mon (Stage 2) INSERT INTO Enrollments (student_id, class_id) VALUES (?, ?); -- Vi pham UNIQUE -> ERROR tu dong</pre>	<pre>// 1. Dang ky tai khoan db.users.insertOne({ _id: personId, email, password, full_name, role: 'student' }) // 2. Teacher tao lop db.classes.insertOne({ course_code, teacher_id, day_of_week, start_time, end_time }) // 3. Dang ky mon db.enrollments.insertOne({ student_id, class_id, status: 'enrolled', payment_requested: false }) // Loi neu trung unique index</pre>

b. DELETE — Xoa du lieu

MySQL — CASCADE + atomic	MongoDB — 2 lenh, co the mat nhut quan
--------------------------	--

<pre>-- 1. Teacher xoa lop hoc DELETE FROM Classes WHERE class_id = ? AND teacher_id = ?; -- CASCADE tu xoa Enrollments lien quan -- 2. Sinh vien huy dang ky DELETE FROM Enrollments WHERE student_id = ? AND class_id = ?; -- 3. Admin reset hoc ky moi TRUNCATE TABLE Enrollments; TRUNCATE TABLE Classes; -- Chay trong 1 transaction, atomic</pre>	<pre>// 1. Teacher xoa lop db.classes.deleteOne({ _id: classId, teacher_id: teacherId }) // PHAI xoa Enrollments thu cong: db.enrollments.deleteMany({class_id: classId}) // 2. Huy dang ky db.enrollments.deleteOne({ student_id, class_id }) // 3. Reset hoc ky (2 lenh riêng) db.enrollments.deleteMany({}) db.classes.deleteMany({}) // KHONG atomic – co the crash</pre>
---	---

c. UPDATE — Cap nhat du lieu

MySQL	MongoDB
<pre>-- 1. Teacher sua gio hoc UPDATE Classes SET day_of_week = ?, start_time = ?, end_time = ? WHERE class_id = ? AND teacher_id = ?; -- 2. Admin xac nhan hoc phi UPDATE Enrollments SET status = 'payed' WHERE student_id = ? AND status = 'enrolled'; -- 3. Admin chuyen stage UPDATE SystemState SET current_stage = 'register_class' WHERE id = 1; -- ENUM tu choi gia tri sai</pre>	<pre>// 1. Sua gio hoc db.classes.updateOne({ _id: classId, teacher_id }, { \$set: { day_of_week, start_time, end_time } }) // 2. Xac nhan hoc phi db.enrollments.updateMany({ student_id, status: 'enrolled' }, { \$set: { status: 'payed' } }) // 3. Chuyen stage db.system_state.updateOne({ _id: 1 }, { \$set: { current_stage: 'register_class' } })</pre>

d. Query dieu kien don — Single Condition SELECT

MySQL — JOIN inline	MongoDB — \$lookup riêng biet
---------------------	-------------------------------

<pre>-- Tim lop theo ma mon SELECT cl.*, u.full_name AS teacher_name FROM Classes cl JOIN Users u ON cl.teacher_id = u.person_id WHERE cl.course_code = 'CS202'; -- Tim lop cua 1 giao vien cu the SELECT cl.*, co.course_name FROM Classes cl JOIN Courses co ON cl.course_code = co.course_code WHERE cl.teacher_id = ?;</pre>	<pre>// Tim lop theo ma mon db.classes.aggregate([{ \$match: { course_code: 'CS202' } }, { \$lookup: { from: "users", localField: "teacher_id", foreignField: "_id", as: "teacher" } }, { \$unwind: "\$teacher" }]) // Tim lop cua 1 GV db.classes.find({ teacher_id: ? })</pre>
---	---

e. Query dieu kien phuc hop — Composite Condition

MySQL	MongoDB
<pre>-- Loc theo Khoa VA So tin chi SELECT * FROM Courses WHERE department = 'CNTT' AND credits = 3; -- Tim lop dang ky co status payed -- va thuoc 1 giao vien cu the SELECT e.*, cl.day_of_week FROM Enrollments e JOIN Classes cl ON e.class_id = cl.class_id WHERE cl.teacher_id = ? AND e.status = 'payed';</pre>	<pre>// Loc theo Khoa VA Tin chi db.courses.find({ \$and: [{ department: 'CNTT' }, { credits: 3 }] }) // Lop payed cua 1 GV db.enrollments.aggregate([{ \$match: { status: 'payed' } }, { \$lookup: { from: "classes", localField: "class_id", foreignField: "_id", as: "cls" } }, { \$match: { "cls.teacher_id": ? } }])</pre>

f. Query voi JOIN — Thoi khoa bieu day du (4 bang)

MySQL — 18 dong, 1 round-trip	MongoDB — 32 dong, pipeline phuc tap hon
-------------------------------	--

<pre>-- TKB sinh vien Stage 4 (chi payed) SELECT co.course_code, co.course_name, co.credits, u.full_name AS teacher_name, cl.day_of_week, cl.start_time, cl.end_time FROM Enrollments e JOIN Classes cl ON e.class_id = cl.class_id JOIN Courses co ON cl.course_code = co.course_code JOIN Users u ON cl.teacher_id = u.person_id WHERE e.student_id = ? AND e.status = 'payed';</pre>	<pre>// MongoDB: 3 stage \$lookup db.enrollments.aggregate([{ \$match: {student_id:?, status:'payed'} }, { \$lookup: { from: "classes", localField: "class_id", foreignField: "_id", as: "cls" } }, { \$unwind: "\$cls" }, { \$lookup: { from: "courses", localField: "cls.course_code", foreignField: "_id", as: "co" } }, { \$unwind: "\$co" }, { \$lookup: { from: "users", localField: "cls.teacher_id", foreignField: "_id", as: "u" } }, { \$unwind: "\$u" }, { \$project: { course_name: "\$co.course_name", credits: "\$co.credits", teacher_name: "\$u.full_name", day: "\$cls.day_of_week" } }])</pre>
---	--

g. Query voi Subquery — Mon chua dang ky (NOT IN)

MySQL — 1 cau SQL, 1 round-trip	MongoDB — 2 lenh, logic o app layer
<pre>-- Mon chua dang ky (Course Catalog) SELECT cl.class_id, co.course_name, co.credits, co.department, u.full_name AS teacher_name, cl.day_of_week, cl.start_time, cl.end_time FROM Classes cl JOIN Courses co ON cl.course_code = co.course_code JOIN Users u ON cl.teacher_id = u.person_id WHERE cl.class_id NOT IN (SELECT class_id FROM Enrollments WHERE student_id = ?);</pre>	<pre>// MongoDB KHONG co subquery native // Buoc 1: fetch ids tai app layer const enrolled = await db .collection("enrollments") .find({ student_id: ? }, { class_id: 1 }) .toArray(); const enrolledIds = enrolled.map(e => e.class_id); // Buoc 2: query voi \$nin db.classes.aggregate([{ \$match: { _id: { \$nin: enrolledIds } } }, { \$lookup: { /* courses */ } }, { \$lookup: { /* users */ } }]) // 2 network round-trips!</pre>

h. Aggregate — Tong hoc phi va dem sinh vien payed

MySQL — SUM/COUNT trong JOIN	MongoDB — \$group pipeline
<pre>-- Tong hoc phi cua sinh vien SELECT SUM(co.credits) * 600000 AS total_fee FROM Enrollments e JOIN Classes cl ON e.class_id = cl.class_id JOIN Courses co ON cl.course_code = co.course_code WHERE e.student_id = ? AND e.status = 'enrolled'; -- Dem SV payed moi lop (Teacher view) SELECT cl.class_id, co.course_name, COUNT(e.student_id) AS paid_students FROM Classes cl JOIN Courses co ON cl.course_code = co.course_code LEFT JOIN Enrollments e ON cl.class_id = e.class_id AND e.status = 'payed' WHERE cl.teacher_id = ? GROUP BY cl.class_id, co.course_name;</pre>	<pre>// Tong hoc phi — can 2 \$lookup db.enrollments.aggregate([{\$match:{student_id:?,status:'enrolled'}}, {\$lookup:{from:"classes", localField:"class_id", foreignField:"_id",as:"cls"}}, {\$unwind:"\$cls"}, {\$lookup:{from:"courses", localField:"cls.course_code", foreignField:"_id",as:"co"}}, {\$unwind:"\$co"}, {\$group:{ _id:null, total:{\$sum:{ \$multiply:["\$co.credits",600000] }} }}]) // Dem SV payed db.enrollments.aggregate([{\$match:{status:'payed'}}, {\$group:{_id:"\$class_id", count:{\$sum:1}}}])</pre>

CHUONG 6

Chung minh MySQL Phu hop hon

7 tieu chi ky thuat — Can cu theo yeu cau thuc te cua he thong

1. Ten giang vien tu dong cap nhat (Referential Integrity)

Yeu cau: Spec ghi ro: "Neu GV cap nhat ho ten, ten hien thi tren danh sach lop cung se tu dong doi theo"

Classes.teacher_id là FK -> Users.person_id. Mọi lần JOIN sẽ lấy full_name hiện tại. Tên đổi ngay, 0 code phụ.

Neu cache teacher_name trong Classes document: can chay updateMany() khi doi ten. De bi bo sot, du lieu bi lenh.

MySQL
vuot troi

2. Ngan dang ky trung lop (UNIQUE Constraint)

Yeu cau: Spec: "Khong duoc phep dang ky mon trung class_id"

UNIQUE KEY (student_id, class_id) tại DB. INSERT thu 2 sẽ bị ERROR ngay, bất kể ứng dụng có kiểm tra hay không.

Unique compound index. Neu không có transaction, 2 request đồng thời có thể vượt qua index check trước khi commit.

MySQL
vuot troi

3. Xac thuc giai doan he thong (ENUM Validation)

Yeu cau: 4 giai doan: create_class, register_class, lock_class, scheduled_class — không thể có giá trị ngoài tap hợp này

ENUM('create_class','register_class','lock_class','scheduled_class') tại DB. Giá trị sai bị từ chối ngay.

String tự do. Typo như "register_Classs" vẫn được lưu. Lỗi sẽ xuất hiện ở runtime, khó debug.

MySQL
vuot troi

4. Reset hoc ky sach (CASCADE DELETE)

Yeu cau: Admin xác nhận học kỳ mới: xóa sạch Classes và Enrollments, giữ nguyên Users và Courses

ON DELETE CASCADE: TRUNCATE Classes -> tự động xóa Enrollments. 1 lệnh, 1 transaction, atomic.

Phải chạy deleteMany(Enrollments) rồi deleteMany(Classes). Server crash giữa chừng = dữ liệu bán vĩnh viễn.

MySQL
vuot troi

5. Subquery NOT IN (Mon chưa đăng ký)

Yêu cầu: Spec mục g: "Su dụng NOT IN hoặc NOT EXISTS với subquery"

WHERE class_id NOT IN (SELECT...) — 1 câu SQL, 1 round-trip, query planner tu động tối ưu.

Không có subquery. Phải: (1) fetch ids về app, (2) truy vấn \$nin. 2 round-trips + logic ở app layer.

MySQL
vuốt trôi

6. JOIN 4 bảng cho TKB (Query với JOIN)

Yêu cầu: Spec mục f: "JOIN Enrollments, Classes, Courses để lấy Tên môn, Tin chi, Giờ học, Tên GV"

1 SELECT với 3 JOIN — 18 dòng, dễ đọc, dễ debug. Query planner chọn index tốt nhất tự động.

3 stage \$lookup lồng nhau — 32 dòng, verbose. Mỗi \$lookup là 1 sub-pipeline, khó maintain.

MySQL
vuốt trôi

7. Aggregate SUM/COUNT (Tính học phí)

Yêu cầu: Spec mục h: "SUM(credits)*600000; COUNT sinh viên paid"

SUM() và COUNT() tích hợp thành vào câu JOIN. Không cần pipeline riêng.

\$group + \$sum + \$multiply + 2 \$lookup = 15+ dòng cho 1 phép tính SUM đơn giản.

MySQL
vuốt trôi

MYSQL LÀ LỰA CHỌN CHÍNH XÁC CHO HỆ THỐNG NÀY MySQL đạt 7/7 tiêu chí đánh giá. Hệ thống đăng ký học phân bổ quan hệ bảng chặt, ràng buộc nghiêm ngặt và nhiều JOIN phức tạp — đây chính xác là bài toán MySQL được thiết kế để giải.

CHUONG 7

Ket luan

Tong ket DSM — MongoDB phu hop khi nao — Khuyen nghi thiet ke

7.1 Tong ket cac quyet dinh DSM trong he thong

Quyet dinh	Ly do chon	Ket qua
MySQL lam RDBMS	Ho tro FK, ENUM, CASCADE native	Dam bao rang buoc
InnoDB engine	ACID, row-level lock, FK support	Nhat quan du lieu
5 bang rieng biet	Chuan hoa (3NF), khong du lieu trung	De bao tri, mo rong
ENUM cho status/stage	DB tu validate, khong the nhap sai	Loai bo bug logic
FK + CASCADE	Reset hoc ky atomic, 1 lenh	An toan, khong data ban
Unique compound key	Ngan dang ky trung tai DB level	Khong race condition
Khong cache ten GV	Lay qua JOIN, luon moi nhat	Nhat quan 100%

7.2 Khi nao MongoDB phu hop hon?

MongoDB khong phai lua chon sai trong moi truong hop. Cac tinh huong MongoDB vuot troi:

Du lieu co cau truc linh hoạt:

Vi du: luu portfolio sinh vien, moi SV co bo cau hoi khac nhau. Schema co dinh cua MySQL gay kho khan.

Can scale ngang (horizontal):

Khi du lieu len toi hang trieu ban ghi va can phan tan ra nhieu server, MongoDB sharding don gian hon MySQL.

Du lieu thay doi cau truc thuong xuyen:

Stage prototype, startup chua co spec ro rang. MongoDB cho phep them/bot truong tuy y.

Log va event streaming:

Luu log hoat dong nguoi dung, audit trail — document model anh xa tu nhien voi cau truc log.

Content management:

Bai viet, san pham, catalog co nhieu loai noi dung khac nhau — document model linh hoạt hon.

Nguyên tắc vàng chọn DB: Du lieu co quan he chat, rang buoc nghiem ngat, nhieu JOIN → Relational (MySQL, PostgreSQL) **Du lieu linh hoat, document-centric, scale ngang, it join** → Document (MongoDB) **He thong nay:** 5 bang, 4 FK, ENUM, CASCADE, 8 loai query voi JOIN va Subquery → MySQL la ap dung chinh xac nhat.

7.3 Khuyen nghi thiet ke cho he thong dang ky hoc phan

- ❶ Giu nguyen schema 5 bang MySQL. Khong them du lieu denormalized (nhu cache ten GV vao Classes).
- ❷ Tat ca bang dung InnoDB engine. Khong dung MyISAM (mat FK support).
- ❸ Them index tren Enrollments(student_id, status) de tang toc query trang Tuition va My Saved.
- ❹ Them index tren Classes(teacher_id) de tang toc query trang Quan ly lop hoc cua Teacher.
- ❺ Dung stored procedure hoac transaction bao gom ca UPDATE SystemState khi chuyen stage.
- ❻ Backup dinh ky bang mysqldump truoc moi lan chuyen Stage. Dac biet truoc khi reset hoc ky.
- ❼ Monitor slow query log. Query TKB (JOIN 4 bang) la query phuc tap nhat, can index day du.

"Data Storage & Management khong chi la viec chon MySQL hay MongoDB.
La viec hieu du lieu cua ban co cau truc the nao, rang buoc gi, quan he ra sao — roi chon cong cu phu hop nhat de dam bao tinh nhat quan, hieu nang va bao tri lau dai."